Paper: PipeDream: Generalized Pipeline Parallelism for DNN Training

Ruilin Li

1. In models with complex connections, like U-Net, how can PipeDream reduce communication between GPUs?
2. In training models with variable-length inputs, like vision-language models, how can PipeDream keep the pipeline balanced?
3. In real-world multi-GPU training, how PipeDream trace errors like NaN losses back to their source in pipeline parallelism architecture?

Aarya Ahjolia

1. How robust is the optimizer's partitioning under changing workloads? (Same with 1F1B-RR)
2. When does a deeper pipeline stop being beneficial? (GPU utilization v/s memory)
3. Would enabling vertical synchronization improve convergence enough to justify its additional coordination costs?

Muzi Zhao

1. Can PipeDream's partitioning algorithm effectively avoid communication bottlenecks when large activations must be transferred across servers?
2. Does higher throughput always mean faster training convergence?
3. Could weight stashing become a memory bottleneck for very large models?

Dhyana Soni

1. Why Traditional Data and Model Parallelism Fail at Scale, and How PipeDream Fixes It
2. How 1F1B Schedules and Automates Work Partitioning
3. Why Pipelining Doesn't Blow Up GPU RAM?

Yuxuan Xu just call me Edward

1. How can PipeDream handle GPU or server failures without significantly reducing pipeline efficiency?
2. Can PipeDream maintain high performance in heterogeneous GPU clusters with different computing and memory capacities?
3. Does PipeDream's higher training throughput necessarily lead to better energy efficiency compared with traditional data parallelism?